runpod
The AI Developer Cloud

Practical AI Inference: From API to Serverless

By Jessica Garson



Hi, I’m Jessica Garson Beauchemin 👋

I lead Developer Relations at Runpod

runpod



The AI Developer Cloud

Experiment, train, fine-tune, deploy, scale on one AI system

runpod.io

runpod
The AI Developer Cloud

What you need to get started

Prerequisites

- A Runpod account. The endpoint runs on an H100 80 GB, and you're billed only while a worker is running.
- A Runpod API key. Create one in the Runpod console under Settings and API Keys.
- npx installed
- An AI coding assistant such as Claude Code
- Python 3 with the requests library. Install it with pip install requests.
- A terminal for sending requests and running the example script. cURL is useful for quick tests. JQ is also used.
- A text editor of your choice for editing the Python code sample.
- Basic familiarity with JSON and HTTP requests.

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

Code from the workshop can be found here

https://github.com/JessicaGarson/practical-ai-inference-workshop

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

There is also Jupyter Notebook to make it easy to follow along with

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

runpod
The AI Developer Cloud

What is inference?

Inference is running a trained model's forward pass on new inputs to produce outputs, with the weights frozen and no gradient computation. It's the deployment phase of a model's lifecycle, where performance is measured in latency, throughput, and cost per request.

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

runpod
The AI Developer Cloud

Runpod basics

Three core products

Public Endpoint

A model Runpod already hosts. You call it with your API key.

Serverless

Your own container on autoscaling GPU workers behind a URL. Workers run only while there's work.

Pod

A GPU or CPU machine you rent and manage directly: start, stop, SSH in, expose ports.

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

runpod
The AI Developer Cloud

I built a game



Two Public Endpoints, one small Python server

Player's browser

Static HTML, JS, and CSS

Interrogate suspects, ask the coroner, make an accusation

Shows a live timer and polls for slow replies every 1.5s

Game server

Runpod Pod, CPU only

Holds the case file and the Runpod API key
Rate limit per player: 10 questions/min, burst of 20
Daily cap: $5 spend and 5,000 jobs (live config)
Portraits generated once, then cached for everyone

Runpod Public Endpoints

Qwen3 32B AWQ: suspect dialogue, coroner, final reveal

FLUX.1 schnell: scene and suspect portraits

Optional: coroner on a dedicated scale-to-zero Serverless endpoint

Browser to server

JSON over HTTPS. Long jobs return a job id; the browser polls /api/.../status until the text is ready.

Server to Runpod

/runsync for quick dialogue and images. /run plus /status for the reveal story and coroner, so the verdict shows instantly.

HOLLOW MANOR

Inference Log 0

Make an Accusation



THE CASE

Halloween night, 1926. Hollow Manor, a crumbling Gothic estate on the moors. Lord Silas Hollow hosted his annual masquerade ball. At 11:58 PM, as the clock struck toward midnight, he was found dead in his library, slumped over his desk beside a half-finished glass of absinthe. The coroner suspects poison. The storm has washed out the only road. The killer is still in the house.

DETECTIVE'S NOTEBOOK

Jot down clues, lies, and contradictions...



Dr. Mortimer Graves
THE CORONER
public endpoint fallback



Lady Vivienne Hollow
THE WIDOW
not yet questioned



Dr. Ambrose Crane
THE PHYSICIAN
not yet questioned



Edgar Finch
THE BUTLER
not yet questioned



Miss Rosalind Vale
THE NIECE
not yet questioned



Madame Zora
THE MEDIUM
not yet questioned

Choose a suspect to begin your interrogation.

server.py

```
def accuse(body):
    suspect = case.SUSPECTS.get(body["suspect"])
    correct = body["suspect"] == case.KILLER    # the code decides, not the LLM

    prompt = case.REVEAL_PROMPT.format(
        solution=case.SOLUTION,                  # the truth never leaves the
server
        accused=suspect["name"],
        verdict="CORRECT" if correct else "WRONG",
        reasoning=body["reasoning"][:800],
    )

    # Verdict now, story later: the reveal runs as an async Runpod job
    job_id = submit_job(LLM_ENDPOINT, chat_input(
        [{"role": "user", "content": prompt}], max_tokens=600))

    return {"correct": correct, "jobId": job_id}
```

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

Three Runpod products run the whole game

Part of the game	Model	Runs on	Why
Suspects and the reveal	Qwen3 32B	Public Endpoint	No GPUs to manage, pay per request
Crime scene and portraits	FLUX.1 schnell	Public Endpoint	4 steps, painted once and cached
The Coroner (optional)	Qwen3 8B on vLLM	Serverless, scale to zero	His cold start is part of the story
Web server	Python, no dependencies	CPU Pod	Keeps the API key and solution server-side

The server keeps the secrets, and Runpod does the inference

Suspects: one synchronous call

Each question is a /runsync request
The last 12 turns go back with it, so suspects remember their lies
The solution never reaches the browser

Coroner and reveal: async jobs

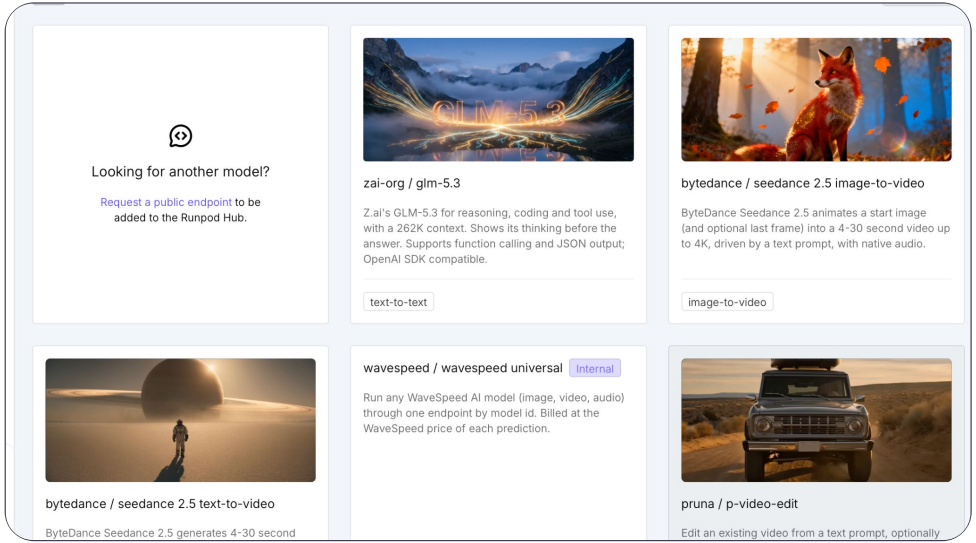
Submitted with /run, then the browser polls job status
The verdict shows instantly while the story is written
Worker health becomes the coffin lid creaking open

runpod
The AI Developer Cloud

Public endpoints

What are Public Endpoints?

Instant API access to pre-deployed AI models for image, video, audio, and text generation. No deployment or infrastructure required. Just create an API key and make a request.



© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

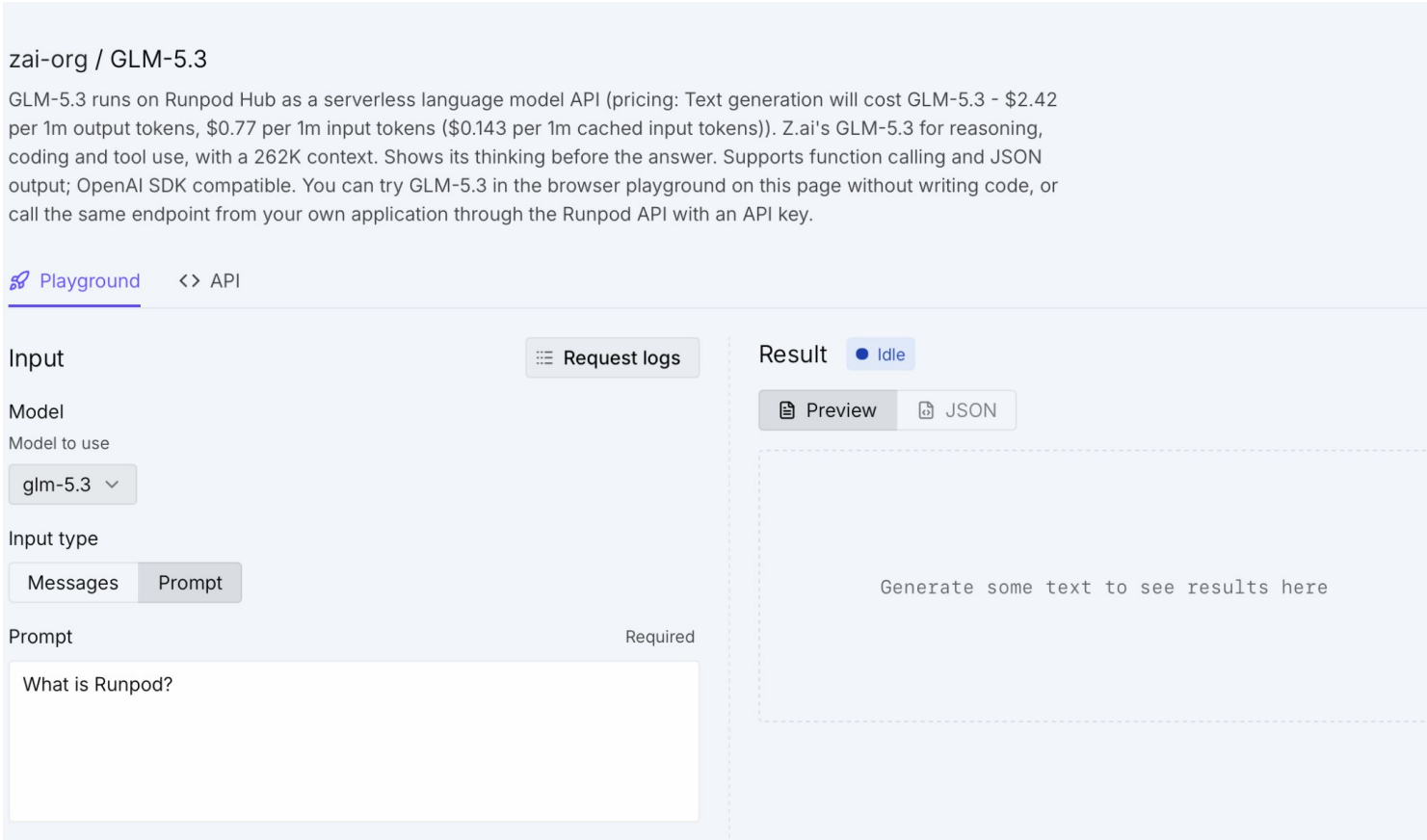
Two types

Synchronous (`/runsync`): Wait for the result and receive it in the response. Best for quick generations.

Asynchronous (`/run`): Receive a job ID immediately and poll for results. Best for longer generations or batch processing.

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

Playground


zai-org / GLM-5.3
GLM-5.3 runs on Runpod Hub as a serverless language model API (pricing: Text generation will cost GLM-5.3 - $2.42 per 1m output tokens, $0.77 per 1m input tokens ($0.143 per 1m cached input tokens)). Z.ai's GLM-5.3 for reasoning, coding and tool use, with a 262K context. Shows its thinking before the answer. Supports function calling and JSON output; OpenAI SDK compatible. You can try GLM-5.3 in the browser playground on this page without writing code, or call the same endpoint from your own application through the Runpod API with an API key.
Playground
API
Input
Request logs
Model
Model to use
glm-5.3
Input type
Messages
Prompt
Prompt
Required
What is Runpod?
Result
Idle
Preview
JSON
Generate some text to see results here

Terminal

```
curl -X POST https://api.runpod.ai/v2/b1dd8hy2hqo1nb/run \
    -H 'Content-Type: application/json' \
    -H 'Authorization: Bearer YOUR_API_KEY' \
    -d '{"input":{"messages":[{"role":"system","content":"You are a helpful
assistant."},{"role":"user","content":"What is Runpod?"}],"sampling_params":
{"max_tokens":4096,"temperature":1},"model":"glm-5.3","reasoning_effort":"low"}}'
```

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

This doesn’t show you the full picture

Let’s look at the revised version

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

Terminal

```python
import os

from openai import OpenAI

client = OpenAI(
    base_url="https://api.runpod.ai/v2/b1dd8hy2hqo1nb/openai/v1",
    api_key=os.environ["RUNPOD_API_KEY"],
)

completion = client.chat.completions.create(
    model="glm-5.3",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "What is Runpod?"},
    ],
    max_tokens=4096,
    temperature=1,
    reasoning_effort="low",
)

print(completion.choices[0].message.content)
```

Open AI example

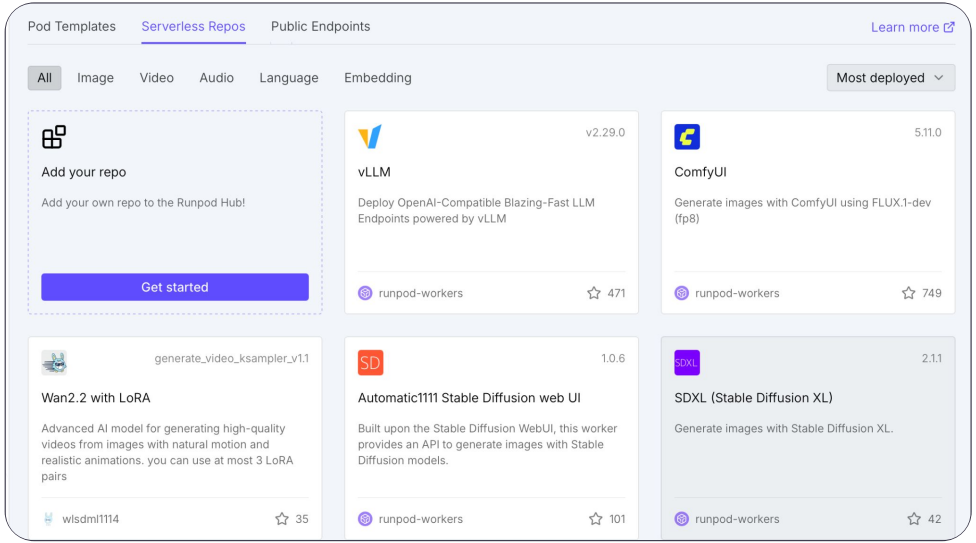
© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

runpod
The AI Developer Cloud

What is Runpod Serverless?

What is Runpod Serverless?

Pay-as-you-go compute for AI models and compute-intensive workloads.



© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

What you need to know about Serverless

Pay as you compute

Compute without servers to manage

You serve AI models for inference, or run other compute-heavy workloads, without provisioning or managing servers.

Pay only while workers run

Workers scale to zero after a short idle timeout when no requests arrive. Startup and model loading are billed too.

Workers run your code in a container

Runpod starts them when requests arrive and stops them when they’re idle.

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

Serverless core concepts

Technical considerations

Endpoints are the access point

Each endpoint gives you a URL to send requests to, with its own GPU, scaling, and other settings.

Two endpoint types

A queue-based endpoint uses a handler function and a request queue. A load balancer endpoint routes requests straight to your own HTTP server, such as FastAPI or Flask.

Cold starts are the tradeoff

A request that arrives with no free worker waits for one to start and load the model. Cached models, FlashBoot, and active workers above zero reduce this.

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

The example we will deploy is OpenJev, a decision model

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

runpod
The AI Developer Cloud

What is a decision model?

Chat models answer in prose you have to parse

Reply 1

"3 stars"

Reply 2

"about a 3"

Reply 3

"I'd say 3 out of 5, because..."

Same question, three formats. Your code has to handle all of them.

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

Decision models answer in the format you ask for

choice

Pick one named option

Returns the choice and a confidence.

noul

Yes or no as a probability

A number from 0 to 1. 0.98 means about 98% confident it is yes.

score

Position on an ordered scale

Returns a score, a legend that maps positions to labels, and a confidence.

runpod
The AI Developer Cloud

What you need to know before you deploy

Zero active workers, so you pay only while one runs

1

max workers

One worker is plenty, and a burst of test requests can't start several GPUs.

0

active workers

The endpoint shuts down when idle.

300 s

idle timeout

The default is 5 s, so every request would pay for another model load.

$4.79

per hour, H100 80 GB flex rate

As of Sept 27, 2026. A few dollars covers this tutorial.

You pay from worker start, including model load

1

Request arrives

Nothing is running yet.

2

Worker starts

A GPU is allocated and billing begins.

3

Model loads

About 30 GB of weights download to the container disk.

4

Requests run

Your cURL and Python calls are answered.

5

Idle for 300 s

Then the worker shuts down and billing stops.

Nothing billed

Billed from worker start until shutdown

Cold starts take minutes, so check readiness first

FlashBoot is on, but it does not speed up the very first boot.

~30 GB

weights download on each cold start

This lab uses no network volume.

~2 min

before a load balancer endpoint returns an error

It does not hold your request while a worker starts.

200

from /v1/version means the model is loaded

"no workers available" is normal. Keep waiting.

Load balancer for a sync API, queue for jobs

Load balancer endpoint

Requests go straight to your web server

One call returns the answer

Errors after about 2 minutes if no worker is ready

Poll /v1/version before you send

Queue-based endpoint

You submit a job and get a job id

A second call returns the result

A queued job waits for a worker to start

Poll /status on the same job id

Check the license and benchmark before you build

84.2%

OpenJev FP8 on its authors' 10,000-question benchmark

TypeSafe's hosted Jev scored 85.4%. Both numbers are the authors' own.

CC BY-NC

license on the OpenJev weights, version 4.0

Non-commercial use, with attribution. For commercial use, open a Hugging Face discussion first.

Apache 2.0

license on the worker code

Decision 2.0 and Qwen3.8-27B keep their upstream licenses.

runpod
The AI Developer Cloud

Deploying

Five settings decide whether the OpenJev deploy works

Setting	Value	Why it matters
Endpoint type	Load balancer, not Queue	A Queue endpoint fails with "not allowed for QB API"
Container image	openjev-worker, pinned by sha256 digest	Pins the tested version (full digest in appendix A1)
Container disk	60 GB	The model weights are about 30 GB
Expose HTTP ports	3000,3001	Skip it and the worker never receives traffic
Environment variables	PORT=3000, PORT_HEALTH=3001, HEALTH_CHECK_PATH=/	Tell the load balancer which ports serve the API and health check

You can do this in the console or with our MCP Server

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

Two ways to connect, and one keeps your API key off disk

Local

Runs on your machine.

Uses a Runpod API key you create and store.

Hosted

```
→ https://mcp.getrunpod.io
```

Point your client at the URL and sign in with Runpod.

Nothing to install, no API key on disk.

How do you authenticate?

We host it for you at https://mcp.getrunpod.io

Point your client at that URL and sign in with Runpod.
Nothing to install, no API key on disk.

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

Terminal

```
npx @runpod/mcp-server@latest add
```

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

Terminal

```
Deploy OpenJev on Runpod Serverless using the image ghcr.io/brandonbondig/openjev-worker@sha256:1f95a727a3de6d0a29808f59d89b49424f4d4bc78c871f5080052abb4105424d (use this exact digest, not the latest tag). Create a load-balancer endpoint (not queue-based) named openjev with:

- One H100 80 GB per worker (OpenJev's measured setup is one 80 GB GPU with native FP8, so no A100 fallback). Look up the H100 GPU pool ID from the Runpod catalog rather than guessing it.
- Environment variables PORT=3000, PORT_HEALTH=3001 and HEALTH_CHECK_PATH=/
- A 60 GB container disk
- Expose HTTP ports 3000 and 3001 in the container configuration
- Active workers 0, maximum workers 1, and an idle timeout of 300 seconds
- Request count scaling (load balancer endpoints don't support queue delay scaling)
- FlashBoot on (the API defaults it to off, unlike the console)

Tell me the hourly price before you create anything. When it's done, give me the endpoint URL and a curl command that sends a test request to `/v1/systemone`.
```

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

runpod
The AI Developer Cloud

Making requests

Setting up environment variables

Terminal

```
export RUNPOD_API_KEY="rpa_yourapikey"
export ENDPOINT_ID="your_endpoint_id"
```

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

Checking readiness

Terminal

```
ready=0
deadline=$(( $(date +%s) + 1200 ))
while [ "$(date +%s)" -lt "$deadline" ]; do
  resp=$(curl -s -m 130 -w '\n%{http_code}' \
    -H "Authorization: Bearer $RUNPOD_API_KEY" \
    "https://$ENDPOINT_ID.api.runpod.ai/v1/version")
  code=${resp##*$'\n'}
  body=${resp%$'\n'*}
  if [ "$code" = "200" ]; then ready=1; echo "Worker ready."; break; fi
  if [ "$code" = "401" ] || [ "$code" = "403" ]; then echo "HTTP $code: check
RUNPOD_API_KEY and its endpoint permissions."; break; fi
  echo "Not ready yet (HTTP $code): ${body:-(no response)}"
  sleep 10
done
[ "$ready" = "1" ] || echo "Worker not ready. Check the messages above and the
endpoint's logs before sending requests."
```

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

Making a cURL request

Terminal

```
curl -sS -m 60 https://$ENDPOINT_ID.api.runpod.ai/v1/systemone \
  -H "Authorization: Bearer $RUNPOD_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"model":"openjev","state":"Review: The sound is great, but the battery barely lasts a day.","questions":{"likes_sound":{"type":"noul","instructions":"Does the reviewer like the sound quality?"}}}' \
  -w "\n"
```

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

Response

Terminal


```
{
   "id": "shim-1790637935267",
   "model": "openjev-FP8 T=0.85 noul=1.829074,0.0 flags={\"perms\":1,\"stagger\":true,
    \"loop_break\":false,\"compact\":false,\"compact_cap\":0,\"layout\":\"\",\"pad\":0,
    \"targeted\":true,\"instr_style\":\"pyrepr\"} shim=shim.py@81a22f1b1b89",
   "answers": {
      "likes_sound": {
         "type": "noul",
         "noul": 0.9801
      }
   },
   "usage": {
      "input_tokens": 75,
      "output_tokens": 0
   }
}

$
```


© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

Let’s take a look at the Python code

github.com/runpod/openjev-serverless/blob/main/examples/article_review.py

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

Python

Terminal

```
python openjev_runpod.py
```

Terminal

```
Worker ready.
Sentiment: mixed (confidence 0.9871)
Recommends: no (probability of yes 0.061)
Rating: 3 stars (score 1.9857, confidence 0.9274)
```

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.

runpod
The AI Developer Cloud

Next steps

If anything comes up along the way or you want to connect with others building with Runpod, join our Discord

discord.com/invite/runpod

© 2026 Runpod Inc. All rights reserved. Confidential and proprietary.


runpod
The AI Developer Cloud